

Experiment No: 4

Aim:- Implement a solution for a Constraint Satisfaction Problem using Branch and Bound and Backtracking for n-queens problem or a graph coloring problem.

Operating System Recommended:- 64-bit Windows OS and Linux

Theory:-

8 Queens Problem using Branch and Bound

The N-Queens problem is a puzzle of placing exactly N queens on an N x N chessboard, such that no two queens can attack each other in that configuration. Thus, no two queens can lie in the same row, column or diagonal.

The branch and bound solution is somehow different, it generates a partial solution until it figures that there's no point going deeper as we would ultimately lead to a dead end.

In the backtracking approach, we maintain an 8x8 binary matrix for keeping track of safe cells (by eliminating the unsafe cells, those that are likely to be attacked) and update it each time we place a new queen. However, it required $O(n^2)$ time to check safe cell and update the queen.

In the 8 queens problem, we ensure the following:

1. No two queens share a row
2. No two queens share a column
3. No two queens share the same left diagonal
4. No two queens share the same right diagonal

We already ensure that the queens do not share the same column by the way we fill out our auxiliary matrix (column by column). Hence, only the left out 3 conditions are left out to be satisfied.

Applying the branch and bound approach

The branch and bound approach suggest that we create a partial solution and use it to ascertain whether we need to continue in a particular direction or not. For this problem, we create 3 arrays to check for conditions 1, 3, and 4.

The Boolean arrays tell which rows and diagonals are already occupied. To achieve this, we need a numbering system to specify which queen is placed.

The indexes on these arrays would help us know which queen we are analysing.

Preprocessing – Create two N x N matrices, one for top-left to bottom- right diagonal, and other for top-right to bottom-left diagonal. We need to fill these in such a way that two queens sharing same top- left bottom-right diagonal will have same value in slash diagonal and two queens sharing same top-right bottom-left diagonal will have same value in backslash Diagonal.

slash diagonal(row)(col) = row + col backslash diagonal(row)(col) = row - col + (N-1) { N = 8 }

{we added (N-1) as we do not need negative values in backslash diagonal}

7	6	5	4	3	2	1	0
8	7	6	5	4	3	2	1
9	8	7	6	5	4	3	2
10	9	8	7	6	5	4	3
11	10	9	8	7	6	5	4
12	11	10	9	8	7	6	5
13	12	11	10	9	8	7	6
14	13	12	11	10	9	8	7

slash diagonal[row][col] = row + col

0	1	2	3	4	5	6	7
1	2	3	4	5	6	7	8
2	3	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	12
6	7	8	9	10	11	12	13
7	8	9	10	11	12	13	14

backslash diagonal[row][col] = row-col+(N-1)

For placing a queen i on row j , check the following:

1. Whether row ' j ' is used or not
2. Whether slash diagonal ' $i + j$ ' is used or not
3. Whether backslash diagonal ' $i - j + 7$ ' is used or not.

If the answer to any one of the following is true, we try another location for queen i on row j , mark the row and diagonal, and recur for queen $i + 1$.

Graph coloring problem's solution using backtracking algorithm Graph coloring

The **graph coloring problem** is to discover whether the nodes of the graph G can be covered in such a way, that no two adjacent nodes have the same color yet only m colors are used. This graph coloring problem is also known as M -colorability decision problem.

The M – colorability optimization problem deals with the smallest integer m for which the graph G can be colored. The integer is known as a chromatic number of the graph.

Here, it can also be noticed that if d is the degree of the given graph, then it can be colored with $d+ 1$ color.

A graph is also known to be planar if and only if it can be drawn in a planar in such a way that no two edges cross each other. A special case is the 4 - colors problem for planar graphs. The problem is to color the region in a map in such a way that no two adjacent regions have the same color. Yet only four colors are needed. This is a problem for which graphs are very useful because a map can be easily transformed into a graph. Each region of the map becomes the node, and if two regions are adjacent, they are joined by an edge.

Graph coloring problem can also be solved using a state space tree, whereby applying a backtracking method required results are obtained.

For solving the **graph coloring problem**, we suppose that the graph is represented by its adjacency matrix $G[1:n, 1:n]$, where, $G[i, j] = 1$ if (i, j) is an edge of G , and $G[i, j] = 0$ otherwise.

The colors are represented by the integers 1, 2, ..., m and the solutions are given by the n -tuple $(x_1, x_2, x_3, \dots, x_n)$, where x_i is the color of node i .

Algorithm for finding the m – colorings of a graph

1. Algorithm mcoloring(k)
2. //this algorithm is formed using the recursive backtracking
3. //schema. The graph is represented by its Boolean adjacency
4. //matrix G [1: n, 1: n]. All assignments of 1, 2, ..., m to the
5. //vertices of the graph such that adjacent vertices are
6. //assigned distinct are printed. K is the index
7. //of the next vertex to color
8. {
9. Repeat
10. {
11. //generate all legal assignments for x[k],
12. Next value (k); // assign to x[k] a legal color.
13. If (x[k] = 0) then return; // no new color possible
14. If (k = n) then // at most m colors have been used to color the n vertices.
15. Write (x [1: n]);
16. Else mcoloring (k + 1)
17. }
18. Until (false);
19. }

This algorithm uses the recursive backtracking schema. In this algorithm colors to be assigned are to determine from the range (0, m), i.e., m colors are available.

The total time required by the above algorithm is **$O(nm^n)$** .

Conclusion:-

Thus, we have implemented n queen problem or a graph coloring problem using Branch and Bound and Backtracking algorithm.

Practical No. 4

Implement a solution for a Constraint Satisfaction Problem using Branch and Bound and Backtracking for n-queens problem or a graph colouring problem.

Program :

```
def print_board(board):
    for row in board:
        print(" ".join("Q" if col else "." for col in row))
    print("\n")

def is_safe(board, row, col, n):
    for i in range(row):
        if board[i][col]:
            return False
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j]:
            return False
    for i, j in zip(range(row, -1, -1), range(col, n)):
        if board[i][j]:
            return False
    return True

def solve_n_queens_backtracking(board, row, n):
    if row == n:
        print_board(board)
        return True
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = True
```

```

        if solve_n_queens_backtracking(board, row + 1, n):
            return True

        board[row][col] = False

    return False

def n_queens_backtracking(n):
    board = [[False] * n for _ in range(n)]

    if not solve_n_queens_backtracking(board, 0, n):
        print("No solution exists.")

def is_safe_branch_and_bound(row, col, cols, diags1, diags2, n):
    return not (cols[col] or diags1[row + col] or diags2[row - col + (n - 1)])

def solve_n_queens_branch_and_bound(row, n, cols, diags1, diags2, board):
    if row == n:
        print_board(board)
        return True

    for col in range(n):
        if is_safe_branch_and_bound(row, col, cols, diags1, diags2, n):
            board[row][col] = True
            cols[col] = True
            diags1[row + col] = True
            diags2[row - col + (n - 1)] = True

            if solve_n_queens_branch_and_bound(row + 1, n, cols, diags1, diags2,
board):
                return True

            board[row][col] = False
            cols[col] = False
            diags1[row + col] = False
            diags2[row - col + (n - 1)] = False

    return False

```

```

def n_queens_branch_and_bound(n):
    board = [[False] * n for _ in range(n)]
    cols = [False] * n
    diags1 = [False] * (2 * n - 1)
    diags2 = [False] * (2 * n - 1)
    if not solve_n_queens_branch_and_bound(0, n, cols, diags1, diags2, board):
        print("No solution exists.")
if __name__ == "__main__":
    n = 5
    print("Backtracking Solution:")
    n_queens_backtracking(n)
    print("\nBranch and Bound Solution:")
    n_queens_branch_and_bound(n)

```

Output :

Backtracking Solution:

```

Q . . . .
. . Q . .
. . . . Q
. Q . . .
. . . Q .

```

Branch and Bound Solution:

```

Q . . . .
. . Q . .
. . . . Q
. Q . . .
. . . Q .

```